

Módulo 9: Evaluación

Text Mining en Python

Antonio Ramón Vázquez Ramírez

2026-09-03

- 1 Carga de datos
 - 1.1 Descripción de los datasets
- 2 Pregunta 1. Text Mining
 - 2.1 Descripción de las columnas
 - 2.2 Se pide
 - 2.2.1 Análisis de Frecuencias
 - 2.2.2 Gráfico de Lemas más comunes
 - 2.2.3 Concurrencias
 - 2.2.4 Reconocimiento de Entidades Nombradas (NER)
 - 2.2.5 Análisis de Casos Específicos
 - 2.2.6 Word Embeddings (Representación Vectorial de Palabras)
- 3 Pregunta 2. Clasificación de Texto: Análisis de sentimiento
 - 3.1 Descripción de las columnas
 - 3.2 Se pide
- 4 Pregunta 3. Grandes Modelos de Lenguaje (LLM)
 - 4.1 Descripción de las columnas
 - 4.2 Se pide
 - 4.2.1 Clustering con BERT
 - 4.2.2 Clustering con Nomic Bert
 - 4.2.3 Clasificación de texto con Qwen3
 - 4.2.4 Comparar la clasificación de Qwen3 con el clustering de BERT
 - 4.2.5 Comparar la clasificación de Qwen3 con el clustering de Nomic Bert
- 5 Pregunta 4. Grandes Modelos de Lenguaje (LLM)
 - 5.1 Páginas web (URL) a procesar
 - 5.2 Se pide
 - 5.2.1 Gemini
 - 5.2.2 Ollama

Carga de entorno python y librerías

1 Carga de datos

Desde la sesión de R, cargamos el archivo `evaluacion.RData`, que contiene tres datasets: `prado`, `reviews` y `documentos`. Exportamos dichos datasets como CSV.

Desde la sesión de R

Hide

```
setwd("G:/Mi unidad/Antonio/estudios/BigData&DataScience/M9/evaluacion")
```

Hide

```
load("evaluacion.RData")

readr::write_delim(prado, "prado.csv", delim = "\t")
readr::write_delim(reviews, "reviews.csv", delim = "\t")
readr::write_delim(documentos, "documentos.csv", delim = "\t")
```

Cargamos los datos.

Hide

```
from pandas import read_csv

prado = read_csv("prado.csv", sep = "\t" , dtype = {"status":str})
reviews = read_csv("reviews.csv", sep = "\t")
documentos = read_csv("documentos.csv", sep = "\t")
```

Comprobamos que los datos se han cargado

[Hide](#)

```
print(f"Tuits de Prado: {len(prado)}")
```

```
## Tuits de Prado: 1038
```

[Hide](#)

```
print(f"Reviews: {len(reviews)}")
```

```
## Reviews: 13000
```

[Hide](#)

```
print(f"Documentos: {len(documentos)}")
```

```
## Documentos: 120
```

1.1 Descripción de los datasets

- **prado:** Cuenta de X (antiguo Twitter) del Museo del Prado: 1.038 tuits (desde 01-04-2025 al 31-07-2026).
- **reviews:** Contiene 13.000 críticas sobre el alojamiento en diversas ciudades de Andalucía.
- **documentos:** Contiene 120 oraciones.

2 Pregunta 1. Text Mining

El dataset `prado` contiene 1.038 tuits, de la cuenta del Museo del Prado.

2.1 Descripción de las columnas

- **status:** Identificador del tuit
- **fecha:** Fecha de publicación del tuit. Desde el 01-04-2025 hasta el 31-07-2026.
- **texto:** El texto del tuit.

Para realizar text mining al texto de los tuits, es importante depurar previamente el texto.

Como, por ejemplo, eliminar del texto: usuarios de X, hashtags, URLs, ...

Para realizar la limpieza del texto, podemos utilizar el siguiente script:

[Hide](#)

```
# === Limpieza de Texto ===

import re, string

prado['textmining'] = prado['texto']

# Quitar "@usuario"
prado['textmining'] = prado['textmining'].str.replace(
    r'@\w+', '', regex=True
)

# Quitar "#hashtag"
prado['textmining'] = prado['textmining'].str.replace(
    r'#\w+', '', regex=True
)

# Quitar "URL"
prado['textmining'] = prado['textmining'].str.replace(
    r'http\S*', '', regex=True
)

# Quitar "e-Mail"
prado['textmining'] = prado['textmining'].str.replace(
    r'\w+@\w+\.\w+', '', regex=True
)

# Si fuera necesario, añadir espacio después de "punto", "coma", "punto y coma", "dos puntos"
prado['textmining'] = prado['textmining'].str.replace(
    r'([.,;:])([a-zA-ZáéíóúñÁÉÍÓÚÑ])', r'\1 \2', regex=True
)

# Conservar letras, dígitos, espacios en blanco, saltos de línea y caracteres de puntuación
prado['textmining'] = prado['textmining'].str.replace(
    r'^\wÃ-ÿ' + re.escape(string.punctuation) + r'\s', '', regex=True
)

# Quitar símbolos
prado['textmining'] = prado['textmining'].str.replace('ï', '')

# Quitar espacios en blanco al inicio y final del texto
prado['textmining'] = prado['textmining'].str.strip()
```

Realizar text mining a los tuits de la cuenta prado , desde 01-06-2025 hasta 31-07-2026.

Utilizar las librerías spacy y nltk , según proceda.

Trabajar con la columna "textmining" . Operar con sustantivos, adjetivos y nombres propios.

2.2 Se pide

- Términos más frecuentes.
- Buscar palabras clave:
 - N-gramas: obtener bigramas, trigramas, ...
 - Colocaciones: obtener bigramas, trigramas, ...
- Visualizar las coocurrencias.
- Reconocimiento de entidades nombradas (NER).
- Podemos clasificar los tuits conforme a una temática determinada. Para extraer temas, podemos basarnos en la información que se desprende del text mining. Se pide:
 - Examinando las palabras clave (bigramas, trigramas, etc) y las coocurrencias obtenidas, indica algunos nombres de pintores y exposiciones.

Si te resulta más cómodo, señala las relaciones oportunas (término1 – término2 – término3 – término4).

- Selecciona aquellos tuits donde figure la cadena de texto "Veronese" . Después, muestra el texto de los cinco primeros tuits (ordenados por fecha, del tuit más antiguo al más reciente). Para finalizar, visualiza las coocurrencias.

- Ídem a la pregunta anterior, pero empleando la cadena de texto "femenino" .
- Word embeddings
 - Modelo Word2Vec. El texto debe estar compuesto únicamente por los lemas deseados.
 - Mostrar las similitudes de los términos: "farnesio" , "caravaggio" , "università" , "falomir" .
 - Visualizar word embeddings mediante el método UMAP.

Empezamos realizando el filtrado temporal que requiere

Hide

```
import pandas as pd
prado["fecha"] = pd.to_datetime(prado["fecha"])
prado_filtrado = prado[(prado["fecha"] >= "2025-06-01") & (prado["fecha"] <= "2026-07-31")].copy()

print(f"Dataset listo. Tuits totales en el rango: {len(prado_filtrado)}")
```

```
## Dataset listo. Tuits totales en el rango: 864
```

2.2.1 Análisis de Frecuencias

Extraemos únicamente los tokens y lemas correspondientes a Sustantivos, Adjetivos y Nombres Propios (NOUN , ADJ , PROPN).

Hide

```
import spacy
from collections import Counter
import matplotlib.pyplot as plt

# Cargar el modelo en español
nlp = spacy.load("es_core_news_sm")

tokens = []
lemas = []

# Procesar con pipe para optimizar recursos
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    for token in doc:
        if token.pos_ in ["NOUN", "ADJ", "PROPN"] and not token.is_stop and not token.is_punct:
            tokens.append(token.text)
            lemas.append(token.lemma_.lower())

# Obtener frecuencias
freq_lemas = Counter(lemas)
df_lemas = pd.DataFrame(freq_lemas.most_common(25), columns=["Lema", "Frecuencia"])

print("Top 10 Lemas:")
```

```
## Top 10 Lemas:
```

Hide

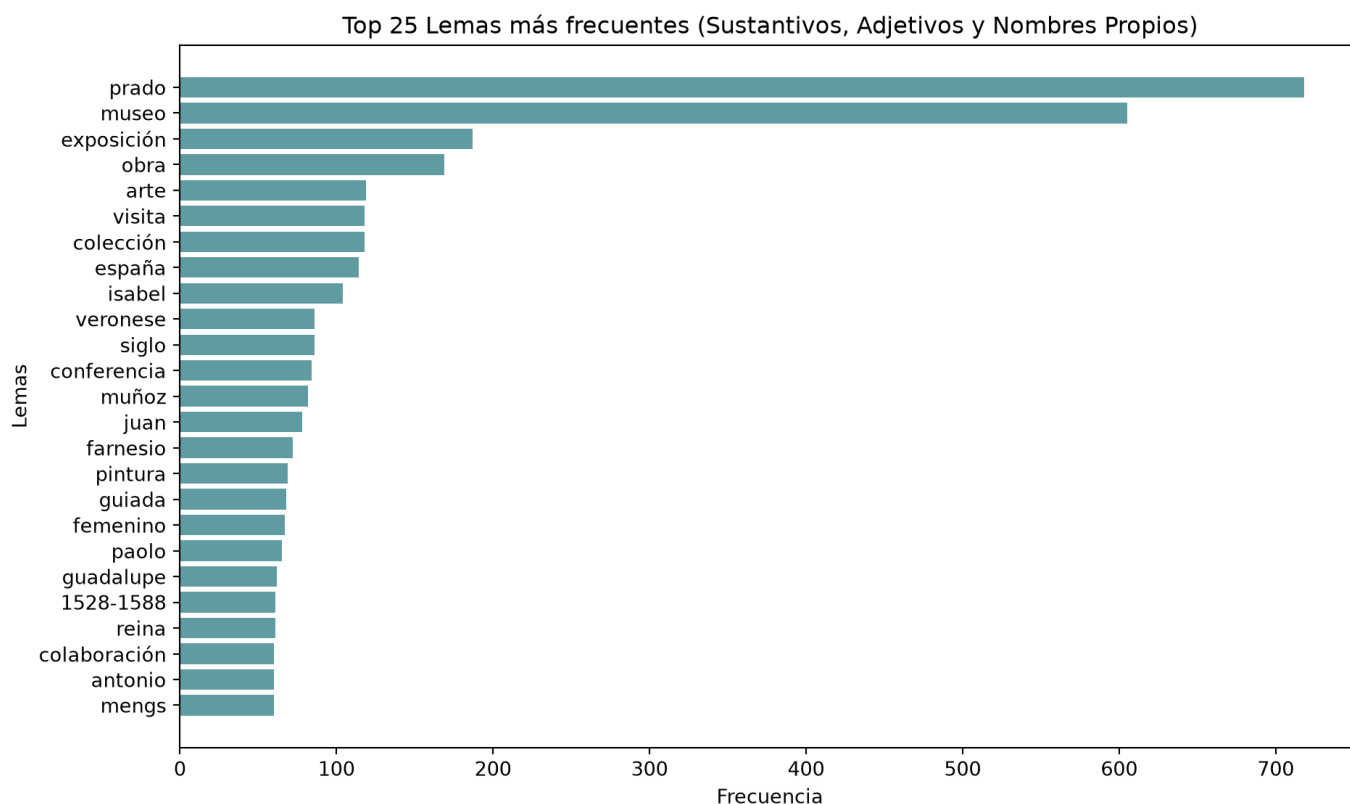
```
print(df_lemas.head(10))
```

##	Lema	Frecuencia
## 0	prado	718
## 1	museo	605
## 2	exposición	187
## 3	obra	169
## 4	arte	119
## 5	visita	118
## 6	colección	118
## 7	españa	114
## 8	isabel	104
## 9	veronese	86

2.2.2 Gráfico de Lemas más comunes

Hide

```
plt.figure(figsize=(10, 6))
plt.barh(df_lemas["Lema"], df_lemas["Frecuencia"], color="cadetblue")
plt.gca().invert_yaxis()
plt.xlabel("Frecuencia")
plt.ylabel("Lemas")
plt.title("Top 25 Lemas más frecuentes (Sustantivos, Adjetivos y Nombres Propios)")
plt.tight_layout()
plt.show()
```



prado (718 apariciones) y museo (605 apariciones): Son, con diferencia, los términos más frecuentes. Lo cual es coherente con el dataset que estamos analizando. Por otro lado, el análisis lingüístico permite desvelar el foco del periodo analizado (2025-2026), destacando los nombres propios 'Isabel' y 'Veronese', dando a entender las principales líneas expositivas y de comunicación del museo en esas fechas.

Como detalle, podemos notar el rango temporal (1528-1588), siendo este el periodo de vida del autor Paolo Veronese.

2.2.3 Concurrencias

Dibujamos las relaciones y fuerzas de enlace entre los términos que aparecen juntos con mayor frecuencia dentro del mismo tuit.

Hide

```

import networkx as nx
import itertools
import pandas as pd
from collections import Counter
import matplotlib.pyplot as plt
coocurrencias = Counter()

# Procesar tuit a tuit para calcular coocurrencias
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    # Extraer lemas únicos de interés por tuit para evitar bucles propios
    lemas_unicos = sorted(list(set([
        token.lemma_.lower() for token in doc
        if token.pos_ in ["NOUN", "ADJ", "PROPN"] and not token.is_stop and not token.is_punct
    ])))
    # Generar pares de combinaciones
    for p1, p2 in itertools.combinations(lemas_unicos, 2):
        coocurrencias[(p1, p2)] += 1

# Generar DataFrame de coocurrencias
df_cooc = pd.DataFrame([{"nodo1": k[0], "nodo2": k[1], "peso": v} for k, v in coocurrencias.items()])
df_cooc = df_cooc.sort_values(by="peso", ascending=False)

# Nos quedamos con las 30 coocurrencias más fuertes para un gráfico limpio
top_cooc = df_cooc.head(30)

# Construir el grafo
G = nx.Graph()
for _, row in top_cooc.iterrows():
    G.add_edge(row["nodo1"], row["nodo2"], weight=row["peso"])

# Dibujar el grafo
plt.figure(figsize=(12, 8))

```

```
## <Figure size 1200x800 with 0 Axes>
```

Hide

```

pos = nx.spring_layout(G, k=0.5, seed=42)

# Configurar nodos
nx.draw_networkx_nodes(G, pos, node_color="skyblue", node_size=1500)

```

```
## <matplotlib.collections.PathCollection object at 0x0000020406D60830>
```

Hide

```

# Configurar aristas según el peso (grosor) de la coocurrencia
edges = G.edges(data=True)
weights = [d['weight'] for u, v, d in edges]
max_weight = max(weights) if weights else 1
edge_widths = [float(w) * 5 / max_weight for w in weights]

nx.draw_networkx_edges(G, pos, width=edge_widths, edge_color="gray", alpha=0.6)

```

```
## <matplotlib.collections.LineCollection object at 0x00000204051A63C0>
```

Hide

```
nx.draw_networkx_labels(G, pos, font_size=10, font_family="sans-serif")
```

```
## {'museo': Text(0.14296550658321736, 0.1252996991671176, 'museo'), 'prado': Text(0.18705902274169245, 0.10112334
345130941, 'prado'), 'obra': Text(0.18813310649254414, 0.16433971304920558, 'obra'), 'colección': Text(0.175320026
94195742, 0.197111330011765, 'colección'), 'exposición': Text(0.009005550973328258, 0.05867916316800223, 'exposici
ón'), 'visita': Text(0.14953046908060108, 0.23931254425709042, 'visita'), 'arte': Text(0.2693874364233922, 0.16139
880294175618, 'arte'), 'paolo': Text(-0.07050941098304231, -0.054155939593636994, 'paolo'), 'veronese': Text(-0.11
872261665910931, 0.014406157686493701, 'veronese'), '1528-1588': Text(-0.12709020449582017, -0.04101495930439203,
'1528-1588'), 'isabel': Text(0.31970157302635915, 0.025141310045026025, 'isabel'), 'farnesio': Text(0.279792422574
87985, -0.03339818285323213, 'farnesio'), 'femenino': Text(0.3558665189980982, 0.09163383193027355, 'femenino'),
'españa': Text(-0.1219623389858652, 0.14116018681766104, 'españa'), 'siglo': Text(0.15431280811384995, 0.001533897
4272519979, 'siglo'), 'juan': Text(-0.9433117272252712, -0.8324243817688048, 'juan'), 'muñoz': Text(-1.0, -0.76712
33651480763, 'muñoz'), 'guiada': Text(0.15052185639918847, 0.4069768487151894, 'guiada')}
```

Hide

```
plt.title("Red de Coocurrencias de Términos (Top 30 conexiones)")
```

```
## Text(0.5, 1.0, 'Red de Coocurrencias de Términos (Top 30 conexiones)')
```

Hide

```
plt.axis("off")
```

```
## (np.float64(-1.1423659844948002), np.float64(0.4982325034928985), np.float64(-0.9625615109696243), np.float64
(0.5371139779160088))
```

Hide

```
plt.tight_layout()
plt.show()
```

Red de Coocurrencias de Términos (Top 30 conexiones)



El análisis visual de la Red de Coocurrencias revela que el núcleo de la red está dominado por los conceptos de la entidad ('museo', 'prado', 'visita guiada'). Sin embargo, en torno a este núcleo, aparecen tres agrupaciones temáticas: en primer lugar, el cluster cerrado que rodea a 'Paolo Veronese (1528-1588)'; en segundo lugar, un cluster que vincula a 'Isabel' (Farnesio) con la perspectiva femenina; y finalmente, un nodo

aislado y sumamente específico dedicado a la obra escultórica de ‘Juan Muñoz’. Este mapa refleja que las redes sociales del museo combinan de manera equilibrada la agenda diaria con grandes hitos de divulgación histórica y contemporánea.

2.2.4 Reconocimiento de Entidades Nombradas (NER)

Utilizamos el reconocedor de entidades de spaCy para extraer nombres de personas, ubicaciones y organizaciones de los tuits.

Hide

```
import pandas as pd
import matplotlib.pyplot as plt

# 1. Extraer entidades con sus etiquetas de tipo
entidades = []
for doc in nlp.pipe(prado_filtrado["textmining"]):
    for ent in doc.ents:
        if ent.label_ in ["PER", "LOC", "ORG"]:
            entidades.append((ent.text.strip(), ent.label_))

# 2. Agrupar, contar frecuencias y ordenar sin funciones auxiliares
df_entidades = pd.DataFrame(entidades, columns=["Entidad", "Tipo"])
df_entidades_grouped = df_entidades.groupby(["Entidad", "Tipo"]).size().reset_index(name="Frecuencia")
top_entidades = df_entidades_grouped.sort_values(by="Frecuencia", ascending=False).head(15)

print("Top 15 Entidades Nombradas más frecuentes:")
```

Top 15 Entidades Nombradas más frecuentes:

Hide

```
print(top_entidades.to_string(index=False))
```

```
##          Entidad Tipo  Frecuencia
##      Museo del Prado LOC         417
##          El Prado LOC           85
##          Juan Muñoz PER           65
##      Paolo Veronese PER           65
##          España LOC            58
##      Isabel de Farnesio PER         49
##          España PER            49
##          Prado LOC            48
##      Guadalupe de México LOC         48
##          Italia LOC            42
## Antonio Raphael Mengs PER            41
##          Goya PER              41
##          Velázquez PER           33
##          Celebrada LOC           30
##          Visitas PER            30
```

De nuevo vemos que el Museo del Prado es la entidad con mayor frecuencia. Sin embargo, el modelo trata “Museo del Prado”, “El Prado” y “Prado” como entidades distintas, lo que muestra las limitaciones del modelo de lenguaje natural.

España sale clasificada como persona, esto puede deberse a la falta de la correcta sintaxis en los tuits donde se menciona a España, denominandola de la misma manera que se haría con una persona, con mayúsculas. Lo mismo puede ocurrir para “Visitas” Celebrada aparece como localización .

Estos hechos evidencian las limitaciones de los modelos pequeños para el correcto análisis de tuits, donde no se siguen las reglas del lenguaje formal

Gráfico de Entidades Nombradas por Categoría

Hide


```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches

# 1. Crear Lienzo explícito
fig, ax = plt.subplots(figsize=(10, 6))

# 2. Configurar mapa de colores según el tipo de entidad
colores_dict = {"PER": "salmon", "LOC": "skyblue", "ORG": "lightgreen"}
colores = [colores_dict.get(tipo, "gray") for tipo in top_entidades["Tipo"]]

# 3. Dibujar barras horizontales
ax.barh(top_entidades["Entidad"], top_entidades["Frecuencia"], color=colores)
```

```
## <BarContainer object of 15 artists>
```

[Hide](#)

```
ax.invert_yaxis()
ax.set_xlabel("Frecuencia")
```

```
## Text(0.5, 0, 'Frecuencia')
```

[Hide](#)

```
ax.set_ylabel("Entidades")
```

```
## Text(0, 0.5, 'Entidades')
```

[Hide](#)

```
# Título limpio (sin texto de Leyenda)
ax.set_title("Top 15 Entidades Nombradas más frecuentes", fontsize=14, weight="bold")
```

```
## Text(0.5, 1.0, 'Top 15 Entidades Nombradas más frecuentes')
```

[Hide](#)

```
# 4. Crear Los elementos visuales de La Leyenda de forma manual
parches_leyenda = [
    mpatches.Patch(color="salmon", label="Persona (PER)"),
    mpatches.Patch(color="skyblue", label="Ubicación (LOC)"),
    mpatches.Patch(color="lightgreen", label="Organización (ORG)")
]

# 5. Añadir La Leyenda al gráfico
# 'loc="lower right"' La coloca abajo a la derecha. Puedes cambiarlo a "upper right" si lo prefieres arriba.
ax.legend(handles=parches_leyenda, loc="lower right", title="Tipos de Entidad", frameon=True, facecolor="white", e
          dgecolor="lightgray")
```

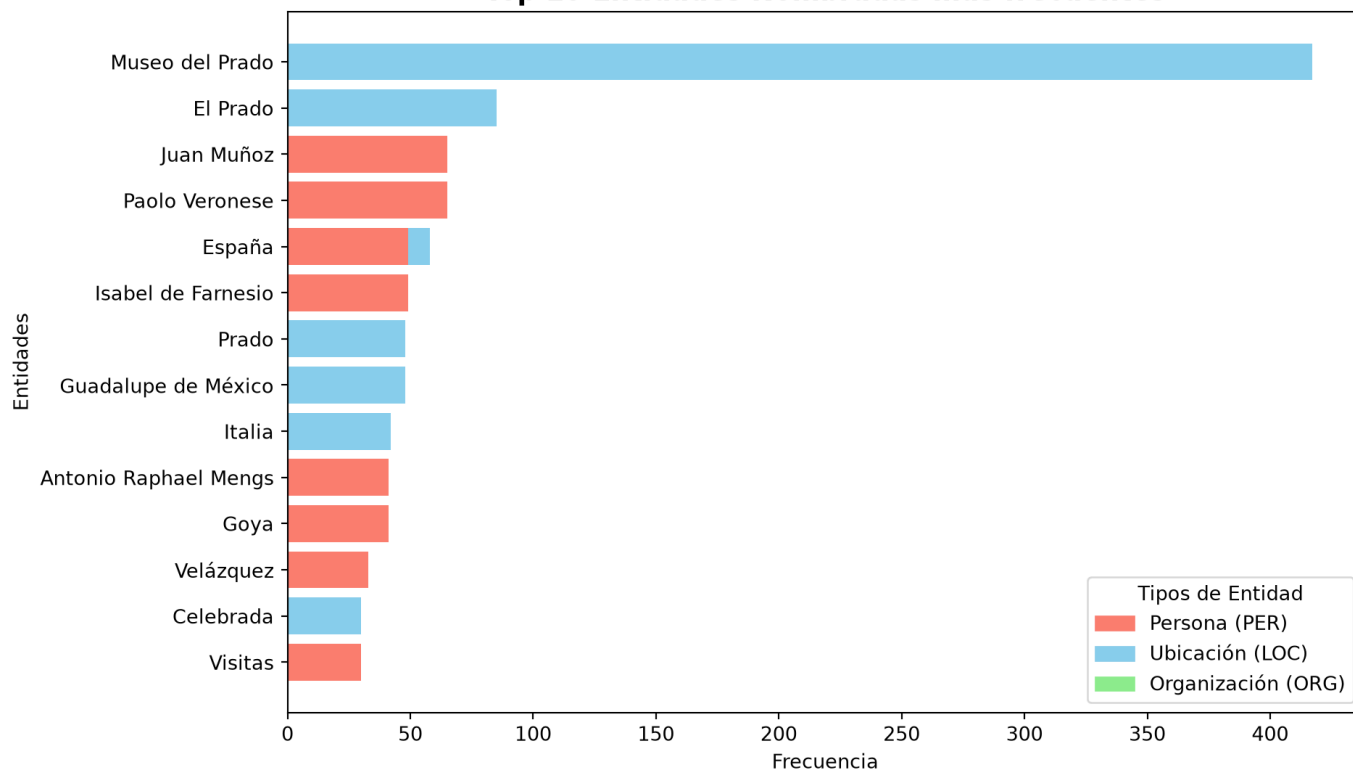
```
## <matplotlib.legend.Legend object at 0x000002040E2ABC00>
```

[Hide](#)

```
plt.tight_layout()

plt.show()
```

Top 15 Entidades Nombradas más frecuentes



2.2.5 Análisis de Casos Específicos

Caso 1: Búsqueda del término “Veronese”

Filtramos los tuits que mencionan “Veronese”, ordenándolos cronológicamente de más antiguo a más reciente, y analizamos sus coocurrencias.

[Hide](#)

```
import networkx as nx
import itertools
import pandas as pd
from collections import Counter
import matplotlib.pyplot as plt

# 1. Filtrar y ordenar cronológicamente
prado_veronese = prado_filtrado[prado_filtrado["textmining"].str.contains("veronese", case=False)].copy()
prado_veronese = prado_veronese.sort_values(by="fecha", ascending=True)

print(f"Total tuits con 'Veronese': {len(prado_veronese)}")
```

```
## Total tuits con 'Veronese': 71
```

[Hide](#)

```
print("\nPrimeros 5 tuits (ordenados cronológicamente):")
```

```
##
## Primeros 5 tuits (ordenados cronológicamente):
```

[Hide](#)

```
for idx, row in prado_veronese.head(5).reset_index().iterrows():
    print(f"{idx+1}. Fecha: {row['fecha'].strftime('%Y-%m-%d')} | Texto: {row['texto']}\n")
```

```
## 1. Fecha: 2025-06-04 | Texto: "Paolo Veronese 1528-1588", catálogo de la exposición comisariada por Miguel Falomir, director del Museo del Prado, y Enrico Maria dal Pozzolo, de la Università degli Studi di Verona. Veronés cierra el ciclo dedicado a la pintura veneciana del Renacimiento https://t.co/yABclXyrvq https://t.co/gRqGC1t88I
##
## 2. Fecha: 2025-06-04 | Texto: #Visitaguiada con Miguel Falomir, comisario de la exposición "Paolo Veronese (1528-1588)". Los días 10, 11, 23 y 24 de junio. Inscripciones abiertas: https://t.co/WVdCILsAHX https://t.co/Jbx9XU9a
ex
##
## 3. Fecha: 2025-06-05 | Texto: #Visitasguiadas a la exposición "Paolo Veronese (1528-1588)". De martes a domingo a las 12:00 h https://t.co/bz8apRw60D https://t.co/G4CXS10k2A
##
## 4. Fecha: 2025-06-06 | Texto: El Museo del Prado abre en horario nocturno la exposición "Paolo Veronese (1528-1588)" este sábado #ElPradodeNoche https://t.co/P5RwoHL553 https://t.co/E2D9y90hSk
##
## 5. Fecha: 2025-06-06 | Texto: #ElPradodeNoche Este sábado 7 podrán visitarse gratuitamente en horario nocturno las exposiciones "Paolo Veronese (1528-1588)" y "Cambio de forma. Mito y metamorfosis en los dibujos romanos de José de Madrazo" https://t.co/XYIngn3DFf
## Colaboran @radio3_rne y @SamsungEspana
```

Hide

```
# 2. Calcular coocurrencias Locales en este filtro
cooc_veronese = Counter()
for doc in nlp.pipe(prado_veronese["textmining"], disable=["ner"]):
    lemas_unicos = sorted(list(set([
        token.lemma_.lower() for token in doc
        if token.pos_ in ["NOUN", "ADJ", "PROPN"] and not token.is_stop and not token.is_punct
    ])))
    for p1, p2 in itertools.combinations(lemas_unicos, 2):
        cooc_veronese[(p1, p2)] += 1

df_cooc_veronese = pd.DataFrame([{"nodo1": k[0], "nodo2": k[1], "peso": v} for k, v in cooc_veronese.items()])
df_cooc_veronese = df_cooc_veronese.sort_values(by="peso", ascending=False)

# 3. Dibujar red de coocurrencias de 'Veronese' (Top 15 conexiones)
top_cooc_veronese = df_cooc_veronese.head(15)
G_veronese = nx.Graph()
for _, row in top_cooc_veronese.iterrows():
    G_veronese.add_edge(row["nodo1"], row["nodo2"], weight=row["peso"])

fig, ax = plt.subplots(figsize=(10, 6))
pos = nx.spring_layout(G_veronese, k=0.6, seed=42)

nx.draw_networkx_nodes(G_veronese, pos, node_color="lightcoral", node_size=1200, ax=ax)
```

```
## <matplotlib.collections.PathCollection object at 0x000002040EF57FE0>
```

Hide

```
nx.draw_networkx_edges(G_veronese, pos, edge_color="gray", alpha=0.5, ax=ax)
```

```
## <matplotlib.collections.LineCollection object at 0x000002040EF56BD0>
```

Hide

```
nx.draw_networkx_labels(G_veronese, pos, font_size=9, font_family="sans-serif", ax=ax)
```

```
## {'paolo': Text(0.17145915817204704, 0.8414908499328622, 'paolo'), 'veronese': Text(0.8222380325950516, 0.247071998865115, 'veronese'), '1528-1588': Text(-0.11832673612247163, -0.8058894504400729, '1528-1588'), 'exposición': Text(-1.0, -0.3593621109609513, 'exposición'), 'prado': Text(0.9843990698671854, -0.800664295066215, 'prado'), 'museo': Text(-0.8597695245118161, 0.877353007669264, 'museo')}
```

Hide

```
ax.set_title("Coocurrencias locales en tuits de 'Veronese' (Top 15)", fontsize=12, weight="bold")
```

```
## Text(0.5, 1.0, "Coocurrencias locales en tuits de 'Veronese' (Top 15)")
```

Hide

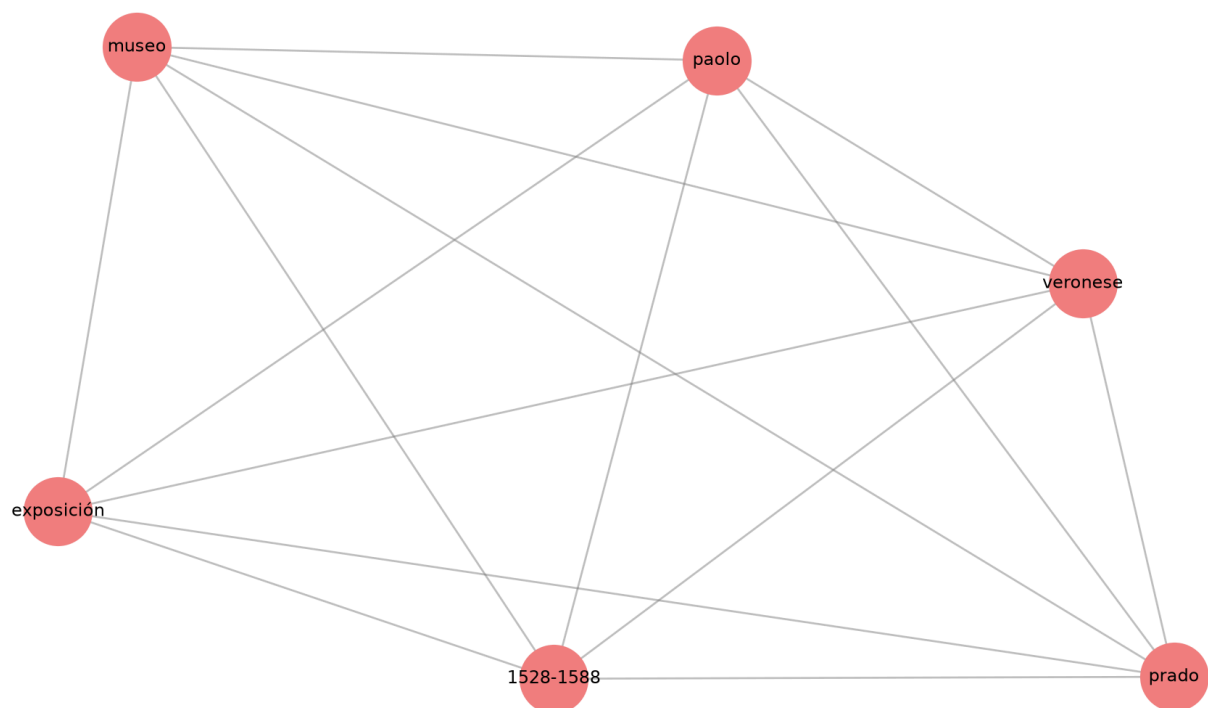
```
ax.axis("off")
```

```
## (np.float64(-1.2083619023360546), np.float64(1.19276097220324), np.float64(-0.9826299085415533), np.float64(1.0540934657707444))
```

Hide

```
plt.tight_layout()
plt.show()
```

Coocurrencias locales en tuits de 'Veronese' (Top 15)



Caso 2: Búsqueda del término “femenino”

Hacemos exactamente el mismo análisis aplicado al concepto “femenino”.

Hide

```
import networkx as nx
import itertools
import pandas as pd
from collections import Counter
import matplotlib.pyplot as plt

# 1. Filtrar y ordenar cronológicamente
prado_femenino = prado_filtrado[prado_filtrado["textmining"].str.contains("femenino", case=False)].copy()
prado_femenino = prado_femenino.sort_values(by="fecha", ascending=True)

print(f"Total tuits con 'femenino': {len(prado_femenino)}")
```

```
## Total tuits con 'femenino': 57
```

Hide

```
print("\nPrimeros 5 tuits (ordenados cronológicamente):")
```

```
##
## Primeros 5 tuits (ordenados cronológicamente):
```

Hide

```
for idx, row in prado_femenino.head(5).reset_index().iterrows():
    print(f"{idx+1}. Fecha: {row['fecha'].strftime('%Y-%m-%d')} | Texto: {row['texto']}\n")
```

```
## 1. Fecha: 2025-08-08 | Texto: Serie documental "El Prado en femenino". Reinas, princesas, regentes y gobernador
as que contribuyeron poderosamente tanto a la creación y consolidación del Museo del Prado como a enriquecer sus c
olecciones https://t.co/KnUjNvnA3G https://t.co/91qf8pcDMz
```

```
##
## 2. Fecha: 2025-08-26 | Texto: La serie documental "El Prado en femenino" cuenta con episodios dedicados a Isabe
l la Católica, María de Hungría, Isabel de Valois, Isabel Clara Eugenia, Isabel de Borbón, Cristina de Suecia o Ma
riana de Austria. Disponible en YouTube https://t.co/Dr8V8DFXbE https://t.co/iPrAffYRnw
```

```
##
## 3. Fecha: 2025-09-01 | Texto: La serie documental "El Prado en femenino" cuenta con episodios dedicados a Isabe
l la Católica, María de Hungría, Isabel de Valois, Isabel Clara Eugenia, Isabel de Borbón, Cristina de Suecia o Ma
riana de Austria. Disponible en YouTube https://t.co/p490QbSs5W #ElPradoEnFemenino https://t.co/2lz5hYNI82
```

```
##
## 4. Fecha: 2025-09-10 | Texto: Serie documental "El Prado en femenino", con episodios dedicados a Isabel la Cató
lica, María de Hungría, Isabel de Valois, Isabel Clara Eugenia, Isabel de Borbón, Cristina de Suecia o Mariana de
Austria. Disponible en YouTube https://t.co/ZBCcdsR7ub #ElPradoEnFemenino https://t.co/rHfVh59C3T
```

```
##
## 5. Fecha: 2025-10-05 | Texto: La serie documental "El Prado en femenino" cuenta con episodios dedicados a Isabe
l la Católica, María de Hungría, Isabel de Valois, Isabel Clara Eugenia, Isabel de Borbón, Cristina de Suecia o Ma
riana de Austria. Disponible en YouTube https://t.co/p490QbSs5W https://t.co/CCDJMOp2Tm
```

Hide

```
# 2. Calcular coocurrencias locales en este filtro
cooc_femenino = Counter()
for doc in nlp.pipe(prado_femenino["textmining"], disable=["ner"]):
    lemas_unicos = sorted(list(set([
        token.lemma_.lower() for token in doc
        if token.pos_ in ["NOUN", "ADJ", "PROPN"] and not token.is_stop and not token.is_punct
    ])))
    for p1, p2 in itertools.combinations(lemas_unicos, 2):
        cooc_femenino[(p1, p2)] += 1
```

```
df_cooc_femenino = pd.DataFrame([{"nodo1": k, "nodo2": k[1], "peso": v} for k, v in cooc_femenino.items()])
df_cooc_femenino = df_cooc_femenino.sort_values(by="peso", ascending=False)
```

```
# 3. Dibujar red de coocurrencias de 'femenino' (Top 15 conexiones)
top_cooc_femenino = df_cooc_femenino.head(15)
G_femenino = nx.Graph()
for _, row in top_cooc_femenino.iterrows():
    G_femenino.add_edge(row["nodo1"], row["nodo2"], weight=row["peso"])
```

```
fig, ax = plt.subplots(figsize=(10, 6))
pos = nx.spring_layout(G_femenino, k=0.6, seed=42)

nx.draw_networkx_nodes(G_femenino, pos, node_color="plum", node_size=1200, ax=ax)
```

```
## <matplotlib.collections.PathCollection object at 0x000002040EED18E0>
```

Hide

```
nx.draw_networkx_edges(G_femenino, pos, edge_color="gray", alpha=0.5, ax=ax)
```

```
## <matplotlib.collections.LineCollection object at 0x00002040EE1DB50>
```

Hide

```
nx.draw_networkx_labels(G_femenino, pos, font_size=9, font_family="sans-serif", ax=ax)
```

```
## ({('femenino', 'prado'): Text(0.5846633444627276, 0.4486110709082114, "{'femenino', 'prado'}"), 'prado': Text(0.6061295129265237, 0.3719326389714494, 'prado'), ('femenino', 'isabel'): Text(-1.0, -0.021166780272535667, "{'femenino', 'isabel'}"), 'isabel': Text(-0.9903300576018575, 0.04836543109140607, 'isabel'), ('isabel', 'prado'): Text(0.6773019175186282, 0.42912629946826014, "{'isabel', 'prado'}"), ('farnesio', 'femenino'): Text(-0.195067153182271, 0.7545444888599675, "{'farnesio', 'femenino'}"), 'femenino': Text(-0.15398149852285956, 0.708024971964719, 'femenino'), ('farnesio', 'prado'): Text(0.5955576725995723, 0.2865234374748059, "{'farnesio', 'prado'}"), ('farnesio', 'isabel'): Text(-0.9552317832492327, 0.11268006143597319, "{'farnesio', 'isabel'}"), ('femenino', 'museo'): Text(0.448947685031365, -0.7879682496265025, "{'femenino', 'museo'}"), 'museo': Text(0.4282080412753657, -0.7480538674158901, 'museo'), ('museo', 'prado'): Text(0.5160111582993615, 0.3859375442614723, "{'museo', 'prado'}"), ('isabel', 'reina'): Text(-0.40756747161571566, -0.06626269849432058, "{'isabel', 'reina'}"), 'reina': Text(-0.42840529581526315, -0.155061452410793, 'reina'), ('femenino', 'reina'): Text(-0.46653006712023887, -0.247159625253608, "{'femenino', 'reina'}"), ('prado', 'reina'): Text(-0.3458849335687266, -0.1953686053024208, "{'prado', 'reina'}"), ('farnesio', 'reina'): Text(-0.5188058910123586, -0.13268735997615244, "{'farnesio', 'reina'}"), ('farnesio', 'museo'): Text(0.49676584969295906, -0.682018237431675, "{'farnesio', 'museo'}"), ('isabel', 'museo'): Text(0.3874846116799149, -0.8442639244130516, "{'isabel', 'museo'}"), ('itinerario', 'prado'): Text(0.720734358202106, 0.33426485616068524, "{'itinerario', 'prado'})})
```

Hide

```
ax.set_title("Coocurrencias locales en tuits de 'femenino' (Top 15)", fontsize=12, weight="bold")
```

```
## Text(0.5, 1.0, "Coocurrencias locales en tuits de 'femenino' (Top 15)")
```

Hide

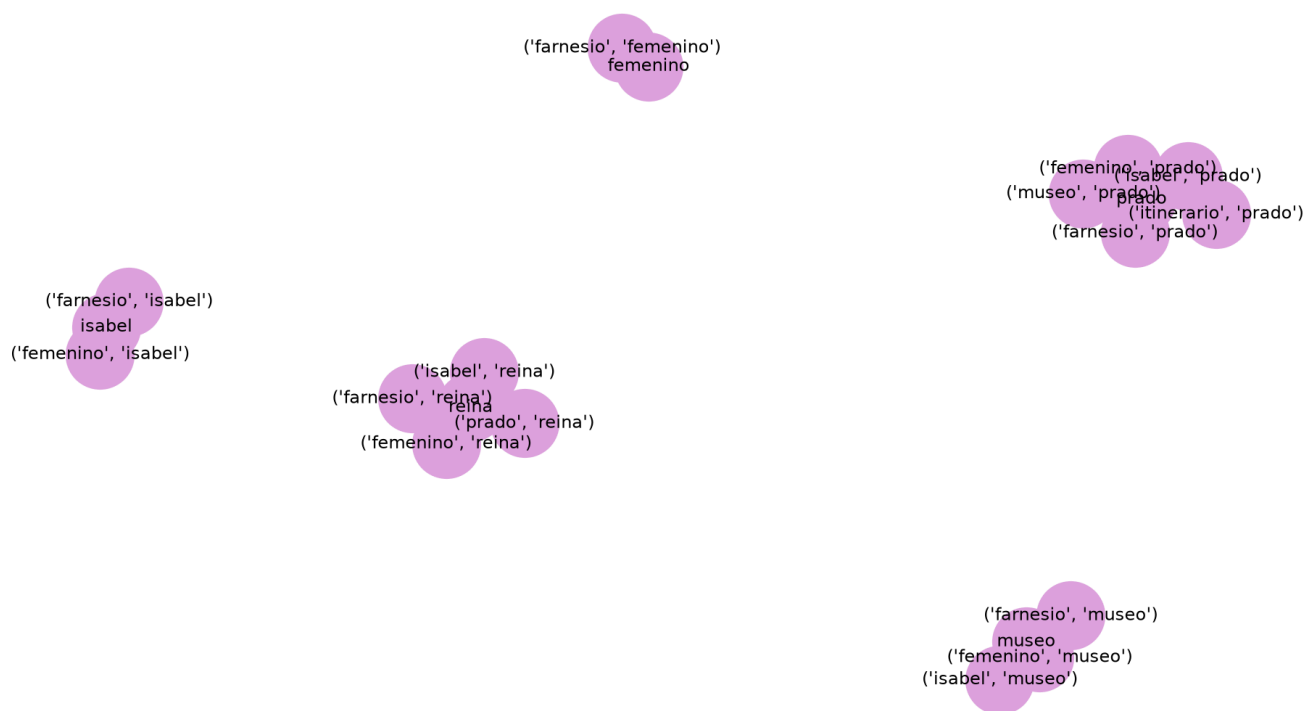
```
ax.axis("off")
```

```
## (np.float64(-1.1806771076112212), np.float64(0.9014114658133272), np.float64(-1.0121388078067186), np.float64(0.9224193722536345))
```

Hide

```
plt.tight_layout()
plt.show()
```

Coocurrencias locales en tuits de 'femenino' (Top 15)



2.2.6 Word Embeddings (Representación Vectorial de Palabras)

Entrenamos un modelo Word2Vec utilizando los lemas válidos de los tuits para extraer similitudes y mapear las palabras en 2D con UMAP.

[Hide](#)

```
from gensim.models import Word2Vec
import numpy as np

# 1. Agrupar lemas por tuit para armar las oraciones de entrenamiento
corpus_lemas = []
for doc in nlp.pipe(prado_filtrado["textmining"], disable=["ner"]):
    lemas_tuit = [token.lemma_lower() for token in doc
                  if token.pos_ in ["NOUN", "ADJ", "PROPN"] and not token.is_stop and not token.is_punct]
    if lemas_tuit:
        corpus_lemas.append(lemas_tuit)

# 2. Entrenar el modelo (min_count=1 para asegurar que todos los términos de examen entren en el vocabulario)
model_w2v = Word2Vec(sentences=corpus_lemas, vector_size=100, window=5, min_count=1, workers=1, seed=42)

# 3. Mostrar similitudes semánticas requeridas
palabras_consulta = ["farnesio", "caravaggio", "università", "falomir"]

for palabra in palabras_consulta:
    print(f"\nTérminos semánticamente más similares a '{palabra}':")
    if palabra in model_w2v.wv:
        similares = model_w2v.wv.most_similar(palabra, topn=5)
        for sim, score in similares:
            print(f"  - {sim}: {score:.4f}")
    else:
        print(f"  - El término '{palabra}' no se encuentra en el modelo.")
```

```
##
## Términos semánticamente más similares a 'farnesio':
##   - prado: 0.9969
##   - reina: 0.9967
##   - museo: 0.9963
##   - vídeo: 0.9960
##   - conferencia: 0.9960
##
## Términos semánticamente más similares a 'caravaggio':
##   - personal: 0.8406
##   - amalia: 0.8264
##   - ingreso: 0.8257
##   - junio: 0.8256
##   - veronese: 0.8251
##
## Términos semánticamente más similares a 'università':
##   - prado: 0.9872
##   - impartido: 0.9871
##   - año: 0.9862
##   - siglo: 0.9861
##   - artista: 0.9856
##
## Términos semánticamente más similares a 'falomir':
##   - prado: 0.9878
##   - visita: 0.9871
##   - museo: 0.9870
##   - isabel: 0.9869
##   - colección: 0.9864
```

Proyección en 2D con UMAP

Hide

```
import umap
import numpy as np
import matplotlib.pyplot as plt

# Palabras clave representativas del corpus para el gráfico
palabras_grafico = ["farnesio", "caravaggio", "università", "falomir", "prado", "museo",
                    "exposición", "arte", "obra", "visita", "isabel", "veronese",
                    "pintor", "cuadro", "madrid", "colección", "donación", "retrato"]

# Filtrar términos presentes en el modelo
palabras_validas = [w for w in palabras_grafico if w in model_w2v.wv]
vectores = np.array([model_w2v.wv[w] for w in palabras_validas])

# Mapear dimensiones a 2D con UMAP
reducer = umap.UMAP(n_neighbors=5, min_dist=0.1, random_state=42)
embedding_2d = reducer.fit_transform(vectores)

# Dibujar el espacio vectorial
fig, ax = plt.subplots(figsize=(10, 8))
ax.scatter(embedding_2d[:, 0], embedding_2d[:, 1], color="coral", edgecolors="black", s=100)
```

```
## <matplotlib.collections.PathCollection object at 0x000020433432960>
```

Hide

```
for i, palabra in enumerate(palabras_validas):
    ax.annotate(palabra, (embedding_2d[i, 0], embedding_2d[i, 1]), xytext=(5, 5), textcoords="offset points", font
                size=11, weight="bold")
```



```
## Text(5, 5, 'farnesio')
## Text(5, 5, 'caravaggio')
## Text(5, 5, 'università')
## Text(5, 5, 'falomir')
## Text(5, 5, 'prado')
## Text(5, 5, 'museo')
## Text(5, 5, 'exposición')
## Text(5, 5, 'arte')
## Text(5, 5, 'obra')
## Text(5, 5, 'visita')
## Text(5, 5, 'isabel')
## Text(5, 5, 'veronese')
## Text(5, 5, 'pintor')
## Text(5, 5, 'cuadro')
## Text(5, 5, 'madrid')
## Text(5, 5, 'colección')
## Text(5, 5, 'donación')
## Text(5, 5, 'retrato')
```

[Hide](#)

```
ax.set_title("Proyección en 2D de Word Embeddings utilizando UMAP", fontsize=14, weight="bold")
```

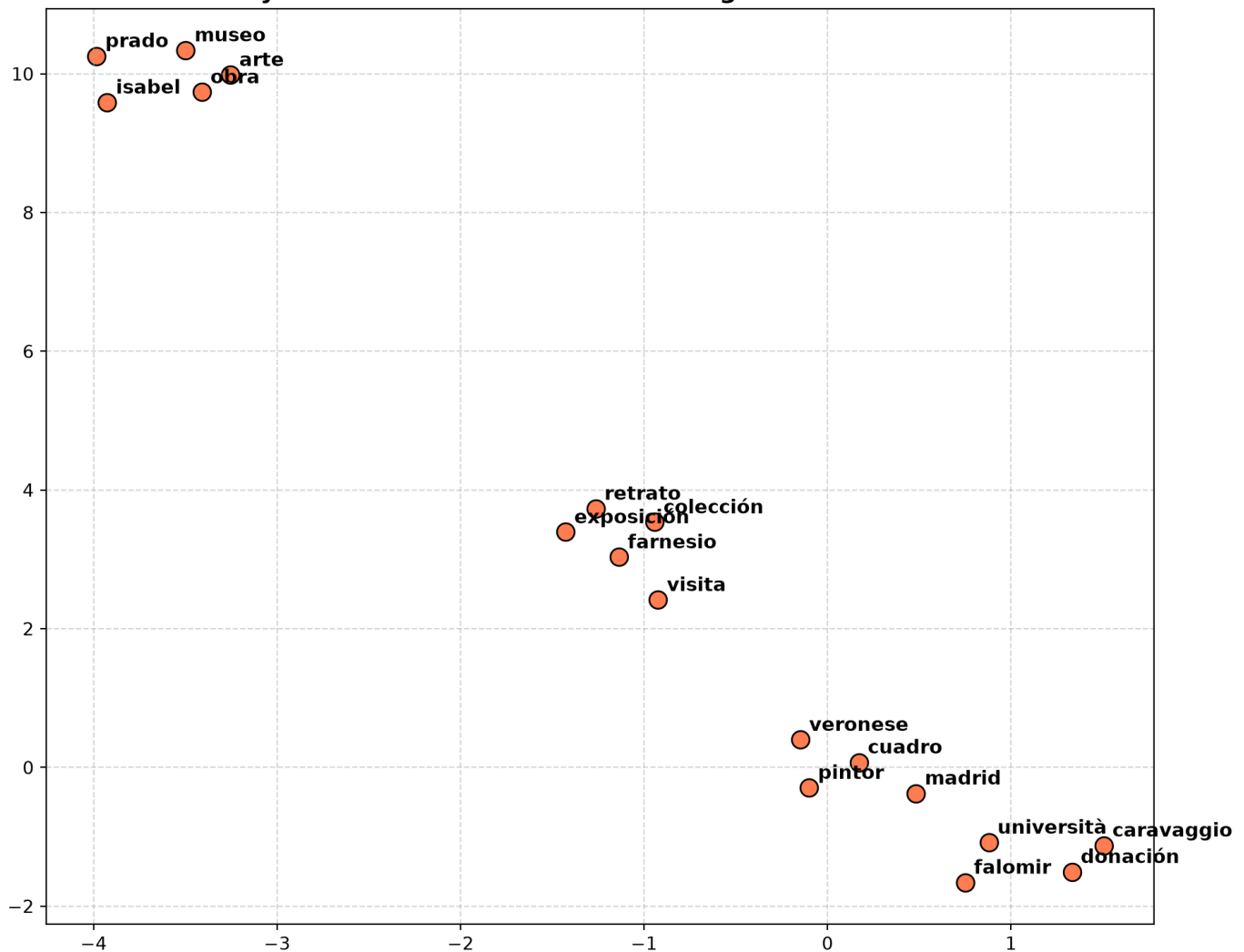
```
## Text(0.5, 1.0, 'Proyección en 2D de Word Embeddings utilizando UMAP')
```

[Hide](#)

```
ax.grid(True, linestyle="--", alpha=0.5)

plt.tight_layout()
plt.show()
```

Proyección en 2D de Word Embeddings utilizando UMAP



3 Pregunta 2. Clasificación de Texto: Análisis de sentimiento

El dataset `reviews` contiene 13.000 críticas sobre el alojamiento en diversas ciudades de Andalucía.

3.1 Descripción de las columnas

- **título:** Breve descripción de la crítica.
- **texto:** El texto de la crítica.
- **puntuacion:** La calificación del usuario, en una escala de 5 estrellas. Variable categórica, con cinco valores posibles: 1 (muy mal), 2 (mal), 3 (regular), 4 (bien), 5 (muy bien).

3.2 Se pide

- Divida `reviews` en dos datasets: dataset de entrenamiento (con el 80% de las filas) y dataset de prueba (con el 20% de las filas restantes). No es necesario que los datasets estén balanceados.
- Mediante la herramienta `fastText`, cree un modelo de clasificación de texto basándose en la variable `puntuacion`.
- En `reviews`, cree una variable categórica, denominada `sentimiento`, siguiendo estas indicaciones:
 - Si `puntuacion` es menor de 3, tomará el valor negativo.
 - Si `puntuacion` es igual a 3, tomará el valor neutro.
 - Si `puntuacion` es mayor de 3, tomará el valor positivo.
- Empleando la herramienta `fastText`, cree un modelo de clasificación de texto basándose en la variable `sentimiento`.
- Utilizando la herramienta `fastText`, cree un modelo de clasificación de texto basándose en la variable `sentimiento`, pero seleccionando únicamente los valores negativo o positivo, es decir, prescindiendo de aquellas filas donde el sentimiento sea neutro.
- Empleando la Regresión Logística, cree un modelo de clasificación de texto basándose en la variable `sentimiento`, pero seleccionando únicamente los valores negativo o positivo.

- Usando el algoritmo Naive Bayes, cree un modelo de clasificación de texto basándose en la variable `sentimiento`, pero seleccionando únicamente los valores negativo o positivo.

4 Pregunta 3. Grandes Modelos de Lenguaje (LLM)

El dataset `documentos` contiene 120 oraciones.

4.1 Descripción de las columnas

- **doc_id**: Identificador del documento.
- **texto**: El texto del documento.

Sabemos de antemano que cada uno de los documentos pertenece a una de estas ocho categorías: Cine, Cocina, Deportes, Economía, Historia, Literatura, Naturaleza, Tecnología. De cada categoría hay 15 documentos.

4.2 Se pide

4.2.1 Clustering con BERT

Empleando Sentence Transformers, utilizar el modelo `hiiamsid/sentence_similarity_spanish_es` para obtener los embeddings de los documentos.

- Visualizar los embeddings mediante UMAP.
- Realizar el clustering, asignando a cada documento una de las ocho categorías anteriormente indicadas.

4.2.2 Clustering con Nomic Bert

Empleando Ollama, usar el modelo `nomic-embed-text-v2-moe` para obtener los embeddings de los documentos.

- Visualizar los embeddings mediante UMAP.
- Realizar el clustering, asignando a cada documento una de las ocho categorías anteriormente señaladas.

4.2.3 Clasificación de texto con Qwen3

Empleando Ollama, usar el modelo `qwen3:4b` para clasificar los documentos, asignando a cada documento una de las ocho categorías anteriormente mencionadas.

Recomendaciones a seguir: No utilizar la salida estructurada, incluir en el prompt las ocho categorías. El thinking debe estar activado.

4.2.4 Comparar la clasificación de Qwen3 con el clustering de BERT

Mostrar aquellos documentos no coincidentes, es decir, la categoría asignada con Qwen3 es distinta al grupo asignado con BERT.

4.2.5 Comparar la clasificación de Qwen3 con el clustering de Nomic Bert

Mostrar aquellos documentos no coincidentes, es decir, la categoría asignada con Qwen3 es distinta al grupo asignado con Nomic Bert.

5 Pregunta 4. Grandes Modelos de Lenguaje (LLM)

Queremos extraer cierta información de varias páginas web de Wikipedia (URL), referentes a 10 personajes ilustres: "Albert Camus", "Marie Curie", "Albert Einstein", "Alexander Fleming", "Mahatma Gandhi", "Gabriel García Márquez", "Nelson Mandela", "Severo Ochoa", "Max Planck" y "Santiago Ramón y Cajal".

5.1 Páginas web (URL) a procesar

Hide

```
urls = [  
    "https://es.wikipedia.org/wiki/Albert_Camus",  
    "https://es.wikipedia.org/wiki/Marie_Curie",  
    "https://es.wikipedia.org/wiki/Albert_Einstein",  
    "https://es.wikipedia.org/wiki/Alexander_Fleming",  
    "https://es.wikipedia.org/wiki/Mahatma_Gandhi",  
    "https://es.wikipedia.org/wiki/Gabriel_García_Márquez",  
    "https://es.wikipedia.org/wiki/Nelson_Mandela",  
    "https://es.wikipedia.org/wiki/Severo_Ochoa",  
    "https://es.wikipedia.org/wiki/Max_Planck",  
    "https://es.wikipedia.org/wiki/Santiago_Ramón_y_Cajal"  
]
```

De cada personaje, queremos obtener la siguiente información:

- **Nombre:** Nombre de la persona.
- **Nacimiento:** Fecha y lugar de nacimiento.
- **Resumen:** Un resumen de su principal logro.
- **Nobel:** Disciplina en la que ganó el Premio Nobel.
- **Año:** Año en que ganó el Premio Nobel.
- **Fallecimiento:** Fecha y lugar de fallecimiento.

Se puede dar el caso, en el que algún personaje tenga más de un premio nobel (Marie Curie).

También, se puede dar el caso, en el que algún personaje nunca fue galardonado con el premio nobel (Mahatma Gandhi).

5.2 Se pide

- Utilizar la API Web fetch de Ollama para extraer el contenido de las páginas web indicadas.

5.2.1 Gemini

Utilizando la librería `chatlas` :

- Definir la salida estructurada a utilizar en el modelo.
- Usar el modelo Gemini para obtener la información requerida.

Es aconsejable hacer una pausa (30 o 45 segundos) entre peticiones, para no saturar el servidor.

5.2.2 Ollama

- Definir la salida estructurada a utilizar en el modelo.
- Usar el modelo `qwen3:4b` para obtener la información requerida.